

第四讲：自动生成词法分析器及其原理

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 3 月 19 日



福州大学
FUZHOU UNIVERSITY

- ① 正则文法与正则表达式
- ② 有限自动机
- ③ 词法分析器生成器
- ④ 小结与作业

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

正则表达式

正则表达式 → 正则文法

正则文法 $\rightarrow$ 正则表达式

④ 小结与作业

式集合中仅含有如下形式的产生式：

$$\alpha \in V_N, \beta = a \vee \beta = aB$$

其中, $a \in V_T$, $B \in V_N$

$$\alpha \rightarrow \beta \quad \alpha \in V_N, \beta = a \vee \beta = aB$$

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

正则表达式

正则表达式 → 正则文法

正则文法 $\rightarrow$ 正则表达式

④ 小结与作业

expression) 是描述正则语言的另一种符号系统,

- 定义：正则表达式 (Regular Expression) 是描述正则语言的另一种符号系统，由字符、运算符构成。

expression) 是描述正则语言的另一种符号系统，

- expression) 是描述正则语言的另一种符号系统，

expression) 是描述正则语言的另一种符号系统，

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

expression) 是描述正则语言的另一种符号系统，

- 定义：正则表达式（Regular Expression）是描述正则语言的另一种符号系统，由字符、运算符构成。
- 核心机制：按照规则匹配字符串。
- 基本运算：
 - 单个字符 a ：匹配字符 a

expression) 是描述正则语言的另一种符号系统，

。 匹配 r_2)。例如：ab 匹配字符串 ab。

- expression) 是描述正则语言的另一种符号系统，
- 。 匹配 r_2)。例如：ab 匹配字符串 ab。

expression) 是描述正则语言的另一种符号系统，

。

配 r2)。例如：ab 匹配字符串 ab。

例如：baa|aab 可以匹配字符串 baa 或者 aab。

- **定义：**正则表达式（Regular Expression）是描述正则语言的另一种符号系统，由字符、运算符构成。
- **核心机制：**按照规则匹配字符串。
- **基本运算：**
 - 单个字符 `a`：匹配字符 `a`
 - `r1r2`：连接（先匹配 `r1` 后匹配 `r2`）。例如：`ab` 匹配字符串 `ab`。
 - `r1|r2`：并（匹配 `r1` 或 `r2`）。例如：`baa|aab` 可以匹配字符串 `baa` 或者 `aab`。

正则表达式

- 定义：正则表达式 (Regular Expression) 是描述正则语言的另一种符号系统，由字符、运算符构成。
- 核心机制：按照规则匹配字符串。
- 基本运算：
 - 单个字符 a ：匹配字符 a
 - r_1r_2 ：连接 (先匹配 r_1 后匹配 r_2)。例如： ab 匹配字符串 ab 。
 - $r_1|r_2$ ：并 (匹配 r_1 或 r_2)。例如： $baa|aab$ 可以匹配字符串 baa 或者 aab 。
 - r^* ：闭包 (匹配 r 0 次或多次)。例如： ab^* 可以匹配字符串 a 、 ab 、 abb 、 $abbb$

正则表达式

- 定义：正则表达式（Regular Expression）是描述正则语言的另一种符号系统，由字符、运算符构成。
- 核心机制：按照规则匹配字符串。
- 基本运算：
 - 单个字符 a：匹配字符 a
 - r1r2：连接（先匹配 r1 后匹配 r2）。例如：ab 匹配字符串 ab。
 - r1|r2：并（匹配 r1 或 r2）。例如：baa|aab 可以匹配字符串 baa 或者 aab。
 - r*：闭包（匹配 r 0 次或多次）。例如：ab* 可以匹配字符串 a、ab、abb、abbb.....
 - 从上面的例子可以看出，闭包优先级最高，其次是连接，最后是并。

正则表达式

- **定义：**正则表达式（Regular Expression）是描述正则语言的另一种符号系统，由字符、运算符构成。
- **核心机制：**按照规则匹配字符串。
- **基本运算：**
 - 单个字符 a ：匹配字符 a
 - r_1r_2 ：连接（先匹配 r_1 后匹配 r_2 ）。例如： ab 匹配字符串 ab 。
 - $r_1|r_2$ ：并（匹配 r_1 或 r_2 ）。例如： $baa|aab$ 可以匹配字符串 baa 或者 aab 。
 - r^* ：闭包（匹配 r 0 次或多次）。例如： ab^* 可以匹配字符串 a 、 ab 、 abb 、 $abbb$
 - 从上面的例子可以看出，闭包优先级最高，其次是连接，最后是并。
- **高级运算（重复类）：**

正则表达式

- 定义：正则表达式 (Regular Expression) 是描述正则语言的另一种符号系统，由字符、运算符构成。
- 核心机制：按照规则匹配字符串。
- 基本运算：
 - 单个字符 a ：匹配字符 a
 - r_1r_2 ：连接 (先匹配 r_1 后匹配 r_2)。例如： ab 匹配字符串 ab 。
 - $r_1|r_2$ ：并 (匹配 r_1 或 r_2)。例如： $baa|aab$ 可以匹配字符串 baa 或者 aab 。
 - r^* ：闭包 (匹配 r 0 次或多次)。例如： ab^* 可以匹配字符串 a 、 ab 、 abb 、 $abbb$
 - 从上面的例子可以看出，闭包优先级最高，其次是连接，最后是并。
- 高级运算 (重复类)：
 - r^+ ：匹配 r 至少 1 次，等价于 rr^* 。

正则表达式

- **定义：**正则表达式 (Regular Expression) 是描述正则语言的另一种符号系统，由字符、运算符构成。
- **核心机制：**按照规则匹配字符串。
- **基本运算：**
 - 单个字符 a ：匹配字符 a
 - r_1r_2 ：连接 (先匹配 r_1 后匹配 r_2)。例如： ab 匹配字符串 ab 。
 - $r_1|r_2$ ：并 (匹配 r_1 或 r_2)。例如： $baa|aab$ 可以匹配字符串 baa 或者 aab 。
 - r^* ：闭包 (匹配 r 0 次或多次)。例如： ab^* 可以匹配字符串 a 、 ab 、 abb 、 $abbb$
 - 从上面的例子可以看出，闭包优先级最高，其次是连接，最后是并。
- **高级运算 (重复类)：**
 - r^+ ：匹配 r 至少 1 次，等价于 rr^* 。
 - $r^?$ ：匹配 r 0 次或 1 次，等价于 $r|\epsilon$ 。
- **等价性：**正则表达式与正则文法等价，可相互转换。

专用符号：

- 在教学中，有一些正则表达式的专用符号：
 - +，等价于|，用于表示并

专用符号：

式

A large, faint, circular watermark of Fuzhou University's logo is centered in the background. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú).

- 在教学中，有一些正则表达式的专用符号：
 - +，等价于|，用于表示并
 - *，表示闭包，但写成上标形式

专用符号：

式

一次或多次

- 在教学中，有一些正则表达式的专用符号：
 - +，等价于|，用于表示并
 - *，表示闭包，但写成上标形式
 - +（上标），类似地，表示匹配一次或多次

专用符号：

式
一次或多次

A large, faint, circular watermark logo of Fuzhou University is centered in the background. It features the university's name in English 'FUZHOU UNIVERSITY' around the top arc and in Chinese '福州大学' around the bottom arc, with the year '1958' at the bottom. In the center of the logo is a large, stylized Chinese character '福' (Fú).

- 在教学中，有一些正则表达式的专用符号：
 - +，等价于|，用于表示并
 - *，表示闭包，但写成上标形式
 - +（上标），类似地，表示匹配一次或多次
 - 用括号改变运算的优先级

见号：
任意小写字母；[a-zA-Z]，表示匹配任意英文字母
.....

- 在工程实践中，有一些常用的记号：
 - 列表记号：[a-z]，表示匹配任意小写字母；[a-zA-Z]，表示匹配任意英文字母；[abc]，表示匹配 a 或 b 或 c；.....

号：
任意小写字母； $[a-zA-Z]$ ，表示匹配任意英文字母
.....
m 次。例如， $ab\{2,4\}$ 可以匹配串 $abb, abbb, abbbb$

- 在工程实践中，有一些常用的记号：
 - 列表记号： $[a-z]$ ，表示匹配任意小写字母； $[a-zA-Z]$ ，表示匹配任意英文字母； $[abc]$ ，表示匹配 a 或 b 或 c ；.....
 - 重复次数： $r\{n,m\}$ ， r 重复 $n\sim m$ 次。例如， $ab\{2,4\}$ 可以匹配串 $abb, abbb, abbbb$ 。

号：
任意小写字母；[a-zA-Z]，表示匹配任意英文字母
.....
n 次。例如，ab{2,4}可以匹配串 abb,abbb,abbbb
先级的括号可以返回匹配的子串，多用于格式化控制

- 在工程实践中，有一些常用的记号：
 - 列表记号：[a-z]，表示匹配任意小写字母；[a-zA-Z]，表示匹配任意英文字母；[abc]，表示匹配 a 或 b 或 c；.....
 - 重复次数：r{n,m}，r 重复 n~m 次。例如，ab{2,4}可以匹配串 abb,abbb,abbbb。
 - 子串提取：没有被用于改变优先级的括号可以返回匹配的子串，多用于格式化提取和替换。

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`
- 匹配示例：
 - `user_name`
 - `123abc` (以数字开头)
 - `u` (长度不足)

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`
- 匹配示例：
 - `user_name`
 - `123abc` (以数字开头)
 - `u` (长度不足)

匹配十六进制颜色码

- 要求：# 后跟 3 或 6 个十六进制字符

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`
- 匹配示例：
 - `user_name`
 - `123abc` (以数字开头)
 - `u` (长度不足)

匹配十六进制颜色码

- 要求: # 后跟 3 或 6 个十六进制字符
- 正则表达式:
#[0-9a-fA-F]{3}){1,2}

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`
- 匹配示例：
 - `user_name`
 - `123abc` (以数字开头)
 - `u` (长度不足)

匹配十六进制颜色码

- 要求: # 后跟 3 或 6 个十六进制字符
- 正则表达式:
#[0-9a-fA-F]{3}){1,2}
- 匹配示例:

匹配合法邮箱前缀

- 要求：仅包含字母/数字/下划线，以字母开头，长度 3-15
- 正则表达式：
`[a-zA-Z][a-zA-Z0-9_]{2,14}`
- 匹配示例：
 - `user_name`
 - `123abc` (以数字开头)
 - `u` (长度不足)

匹配十六进制颜色码

- 要求：# 后跟 3 或 6 个十六进制字符
- 正则表达式：
`#([0-9a-fA-F]{3}){1,2}`
- 匹配示例：
 - #FFF
 - #FF00CC
 - #FOG (包含非十六进制字符)

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

正则表达式

正则表达式 → 正则文法

正则文法 $\rightarrow$ 正则表达式

④ 小结与作业

的基本操作，为每个子表达式创建非终结符，这
 $\rightarrow aB$ 或 $A \rightarrow a$)。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

的基本操作，为每个子表达式创建非终结符，递归地构造非终结符串 aB 或 $A \rightarrow a$ 。

- **核心思想**：将正则表达式分解为基本操作，为每个子表达式创建非终结符，递归生成右线性文法（形式： $A \rightarrow aB$ 或 $A \rightarrow a$ ）。
- **转换规则**：

的基本操作，为每个子表达式创建非终结符，递归地构造非终结符串 aB 或 $A \rightarrow a$ 。

- 的基本操作，为每个子表达式创建非终结符，递归地构造非终结符串 aB 或 $A \rightarrow a$ 。

的基本操作，为每个子表达式创建非终结符，递归地构造非终结符串 aB 或 $A \rightarrow a$ 。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

基本操作，为每个子表达式创建非终结符，递归地构造非终结符串 aB 或 $A \rightarrow a$ 。

- **核心思想**：将正则表达式分解为基本操作，为每个子表达式创建非终结符，递归生成右线性文法（形式： $A \rightarrow aB$ 或 $A \rightarrow a$ ）。
- **转换规则**：
 - ① 单个字符 a ： $S \rightarrow a$
 - ② 并 $R_1 | R_2$ ： $S \rightarrow S_{R_1} | S_{R_2}$
 - ③ 连接 $R_1 R_2$ ： $S \rightarrow S_{R_1} B$ ，其中 B 为 R_2 的开始符号（需确保 $S_{R_1} B$ 符合右线性）

基本操作，为每个子表达式创建非终结符，这
 aB 或 $A \rightarrow a$)。

B 为 R_2 的开始符号（需确保 $S_{R_1} B$ 符合右线性）

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

基本操作，为每个子表达式创建非终结符，递归地构造文法（如 $A \rightarrow aB$ 或 $A \rightarrow a$ ）。

B 为 R_2 的开始符号（需确保 $S_{R_1}B$ 符合右线性）

线性文法（仅允许 $A \rightarrow aB$ 或 $A \rightarrow a$ 形式）

- **核心思想**：将正则表达式分解为基本操作，为每个子表达式创建非终结符，递归生成右线性文法（形式： $A \rightarrow aB$ 或 $A \rightarrow a$ ）。
- **转换规则**：
 - ① 单个字符 a ： $S \rightarrow a$
 - ② 并 $R_1 | R_2$ ： $S \rightarrow S_{R_1} | S_{R_2}$
 - ③ 连接 $R_1 R_2$ ： $S \rightarrow S_{R_1} B$ ，其中 B 为 R_2 的开始符号（需确保 $S_{R_1} B$ 符合右线性）
 - ④ 闭包 R^* ： $S \rightarrow S_R S | \varepsilon$
- **关键约束**：最终文法必须是 **右线性文法**（仅允许 $A \rightarrow aB$ 或 $A \rightarrow a$ 形式）

基本操作，为每个子表达式创建非终结符，这
 aB 或 $A \rightarrow a$)。

B 为 R_2 的开始符号（需确保 $S_{R_1} B$ 符合右线性）

线性文法（仅允许 $A \rightarrow aB$ 或 $A \rightarrow a$ 形式）
 允许 $A \rightarrow abcdeB$ 这种形式

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍

- 正则表达式: a^*b
- 转换步骤:

- 授课教师：柯嵩宇

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

号 ε 替换为 b (避免两个非终结符相邻)

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \varepsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ε 替换为 b (避免两个非终结符相邻)

号 ε 替换为 b (避免两个非终结符相邻)
线性!)

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \epsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ϵ 替换为 b (避免两个非终结符相邻)
 - ③ 生成文法: $S \rightarrow aS \mid b$ (右线性!)

号 ε 替换为 b (避免两个非终结符相邻)
线性!)

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \epsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ϵ 替换为 b (避免两个非终结符相邻)
 - ③ 生成文法: $S \rightarrow aS \mid b$ (右线性!)
- 文法验证:

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \varepsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ε 替换为 b (避免两个非终结符相邻)
 - ③ 生成文法: $S \rightarrow aS \mid b$ (右线性!)
- 文法验证:
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots \Rightarrow a^nS \Rightarrow a^nb$

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \varepsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ε 替换为 b (避免两个非终结符相邻)
 - ③ 生成文法: $S \rightarrow aS \mid b$ (右线性!)
- 文法验证:
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots \Rightarrow a^n S \Rightarrow a^n b$

号 ε 替换为 b (避免两个非终结符相邻)
(线性!)

$\Rightarrow a^n b$

- 正则表达式: a^*b
- 转换步骤:
 - ① a^* 部分: $A \rightarrow aA \mid \varepsilon$
 - ② $a^*b = a^* \cdot b$: 将 a^* 的结束符号 ε 替换为 b (避免两个非终结符相邻)
 - ③ 生成文法: $S \rightarrow aS \mid b$ (右线性!)
- 文法验证:
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots \Rightarrow a^nS \Rightarrow a^nb$
 - $L(G) = \{a^nb \mid n \geq 0\} = a^*b \checkmark$

号 ε 替换为 b (避免两个非终结符相邻)
(线性!)

$\Rightarrow a^n b$

(开始符号) $\rightarrow$ 违反右线性!

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

转换示例 2: $(a|b)^*a$

- 正则表达式: $(a|b)^*a$



- 正则表达式: $(a|b)^*a$
- 转换步骤:
 - ① $(a|b)^*$ 部分: $A \rightarrow aA \mid bA \mid \varepsilon$

- 授课教师：柯嵩宇 福州大学计算机与大数据学院
- 编译原理 13 / 86

ε 替换为 a (避免 Aa)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

ε 替换为 a (避免 Aa)
(右线性!)

- 正则表达式: $(a|b)^*a$
- 转换步骤:
 - ① $(a|b)^*$ 部分: $A \rightarrow aA \mid bA \mid \varepsilon$
 - ② $(a|b)^*a = (a|b)^* \cdot a$: 将 A 的 ε 替换为 a (避免 Aa)
 - ③ 生成文法: $S \rightarrow aS \mid bS \mid a$ (右线性!)

ε 替换为 a (避免 Aa)
(右线性!)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

ε 替换为 a (避免 Aa)
(右线性!)

$\Rightarrow a^n a = a^{n+1}$

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

ε 替换为 a (避免 Aa)
(右线性!)

$\Rightarrow a^n a = a^{n+1}$

- ε 替换为 a (避免 Aa)
(右线性!)
- $\Rightarrow a^n a = a^{n+1}$

- 正则表达式: $(a|b)^*a$
- 转换步骤:
 - ① $(a|b)^*$ 部分: $A \rightarrow aA \mid bA \mid \varepsilon$
 - ② $(a|b)^*a = (a|b)^* \cdot a$: 将 A 的 ε 替换为 a (避免 Aa)
 - ③ 生成文法: $S \rightarrow aS \mid bS \mid a$ (右线性!)
- 文法验证:
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots \Rightarrow a^n S \Rightarrow a^n a = a^{n+1}$
 - $S \Rightarrow bS \Rightarrow \dots \Rightarrow b^m a$
 - $L(G) = \{wa \mid w \in \{a, b\}^*\} = (a|b)^*a \checkmark$

- 正则表达式: $(a|b)^*a$
- 转换步骤:
 - ① $(a|b)^*$ 部分: $A \rightarrow aA \mid bA \mid \varepsilon$
 - ② $(a|b)^*a = (a|b)^* \cdot a$: 将 A 的 ε 替换为 a (避免 Aa)
 - ③ 生成文法: $S \rightarrow aS \mid bS \mid a$ (右线性!)
- 文法验证:
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots \Rightarrow a^n S \Rightarrow a^n a = a^{n+1}$
 - $S \Rightarrow bS \Rightarrow \dots \Rightarrow b^m a$
 - $L(G) = \{wa \mid w \in \{a, b\}^*\} = (a|b)^*a \checkmark$
- 闭包 R^* 的转换是关键, 需用 $S \rightarrow S_R S \mid \varepsilon$ 替代 $S \rightarrow S_R S$ (避免无限循环)

- 13 / 86

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- 正则表达式: ab^*c
- 转换步骤:
 - ① b^* 部分: $B \rightarrow bB \mid \varepsilon$
 - ② $ab^*c = a \cdot b^* \cdot c$: 分步连接
 - $b^* \cdot c$: $B \rightarrow bB \mid c$

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- 正则表达式: ab^*c
- 转换步骤:
 - ① b^* 部分: $B \rightarrow bB \mid \varepsilon$
 - ② $ab^*c = a \cdot b^* \cdot c$: 分步连接
 - $b^* \cdot c$: $B \rightarrow bB \mid c$
 - $a \cdot B$: $S \rightarrow aB$

A large, faint, circular watermark of the Fuzhou University logo is centered on the slide. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú).

- 正则表达式: ab^*c
- 转换步骤:
 - ① b^* 部分: $B \rightarrow bB \mid \varepsilon$
 - ② $ab^*c = a \cdot b^* \cdot c$: 分步连接
 - $b^* \cdot c$: $B \rightarrow bB \mid c$
 - $a \cdot B$: $S \rightarrow aB$
 - ③ 生成文法: $S \rightarrow aB \quad B \rightarrow bB \mid c$

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ↺ 🔍 ↻

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$S \mid \epsilon$

- 关键技巧：
 - 避免非终结符相邻：用中间非终结符拆分（如 $S \rightarrow aB, B \rightarrow c$ ）
 - 闭包优先处理： $R^* \rightarrow S \rightarrow S_R S \mid \varepsilon$

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$\epsilon S \mid \epsilon$

- 关键技巧：
 - 避免非终结符相邻：用中间非终结符拆分（如 $S \rightarrow aB, B \rightarrow c$ ）
 - 闭包优先处理： $R^* \rightarrow S \rightarrow S_R S \mid \varepsilon$
 - 单字符直接处理： $a \rightarrow S \rightarrow a$

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$\rightarrow S \mid \varepsilon$

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$S \mid \varepsilon$

B 或 $A \rightarrow a$

- ① 检查所有产生式是否为 $A \rightarrow aB$ 或 $A \rightarrow a$

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$S \mid \varepsilon$

B 或 $A \rightarrow a$

验证 b, ab, abb)

- 终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)
- $S \mid \varepsilon$
- B 或 $A \rightarrow a$
- 验证 b, ab, abb)

终结符拆分 (如 $S \rightarrow aB, B \rightarrow c$)

$S \mid \epsilon$

B 或 $A \rightarrow a$

验证 b, ab, abb)

止条件 $\mid \epsilon$)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍

- ① 正则文法与正则表达式
 - 正则表达式
 - 正则表达式 $\rightarrow$ 正则文法
 - 正则文法 $\rightarrow$ 正则表达式
- ② 有限自动机
- ③ 词法分析器生成器
- ④ 小结与作业

转换原理

- **核心思想：**将右线性文法转换为方程组，利用 Arden 定理求解正则表达式。



力方程组，利用 **Arden 定理** 求解正则表达式。

- **核心思想：** 将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- **关键约束：**

方程组，利用 **Arden 定理** 求解正则表达式。

aB 或 $A \rightarrow a$)

- 核心思想：将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- 关键约束：
 - 文法必须是右线性文法 ($A \rightarrow aB$ 或 $A \rightarrow a$)

方程组，利用 **Arden 定理** 求解正则表达式。

aB 或 $A \rightarrow a)$

$|\alpha_2| \cdots |\alpha_k$

- 核心思想：将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- 关键约束：
 - 文法必须是**右线性文法** ($A \rightarrow aB$ 或 $A \rightarrow a$)
 - 产生式需转换为方程： $A = \alpha_1 | \alpha_2 | \dots | \alpha_k$

方程组，利用 **Arden 定理** 求解正则表达式。

aB 或 $A \rightarrow a$)
 $|\alpha_2| \cdots |\alpha_k$

的解为 $X = A^*B$ (A 和 B 为正则表达式)

- **核心思想**：将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- **关键约束**：
 - 文法必须是**右线性文法** ($A \rightarrow aB$ 或 $A \rightarrow a$)
 - 产生式需转换为方程： $A = \alpha_1 | \alpha_2 | \dots | \alpha_k$
- **Arden 定理**：方程 $X = AX + B$ 的解为 $X = A^*B$ (A 和 B 为正则表达式)

方程组，利用 **Arden 定理** 求解正则表达式。

aB 或 $A \rightarrow a$
 $\alpha_2 | \cdots | \alpha_k$

的解为 $X = A^*B$ (A 和 B 为正则表达式)
(需特殊处理)

- **核心思想**：将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- **关键约束**：
 - 文法必须是**右线性文法** ($A \rightarrow aB$ 或 $A \rightarrow a$)
 - 产生式需转换为方程： $A = \alpha_1 | \alpha_2 | \dots | \alpha_k$
- **Arden 定理**：方程 $X = AX + B$ 的解为 $X = A^*B$ (A 和 B 为正则表达式)
 - **关键**： A 不能包含 ϵ (若包含需特殊处理)

方程组，利用 **Arden 定理** 求解正则表达式。

aB 或 $A \rightarrow a$)
 $|\alpha_2| \cdots |\alpha_k$

的解为 $X = A^*B$ (A 和 B 为正则表达式)
需特殊处理)

- **核心思想**：将右线性文法转换为方程组，利用 **Arden 定理** 求解正则表达式。
- **关键约束**：
 - 文法必须是**右线性文法** ($A \rightarrow aB$ 或 $A \rightarrow a$)
 - 产生式需转换为方程： $A = \alpha_1 | \alpha_2 | \dots | \alpha_k$
- **Arden 定理**：方程 $X = AX + B$ 的解为 $X = A^*B$ (A 和 B 为正则表达式)
 - **关键**： A 不能包含 ϵ (若包含需特殊处理)
 - **例**： $X = aX + b \Rightarrow X = a^*b$

二方程

A large, faint, circular watermark of the Fuzhou University logo is centered in the background. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top and the Chinese characters "福州大学" around the bottom, with the year "1958" in the center. In the middle of the logo is a large, stylized Chinese character "福" (Fú). At the bottom right of the slide, there is a small navigation bar with several icons: a square, a circle, a triangle, and some symbols resembling arrows or document icons.

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

① 写出方程：为每个非终结符建立方程

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

② 代入求解：用子方程替换父方程

$$S = a(\underbrace{aS + b}_A) + b = aaS + ab + b$$
$$S = (aa)^*(ab + b) \quad (\text{应用 Arden 定理})$$

福州大学计算机与大数据学院

18 / 86

① 写出方程：为每个非终结符建立方程

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

② 代入求解：用子方程替换父方程

$$S = a(\underbrace{aS + b}_A) + b = aaS + ab + b$$
$$S = (aa)^*(ab + b) \quad (\text{应用 Arden 定理})$$

③ 化简表达式： $S = (aa)^*(a + \varepsilon)b$ (因 $ab + b = (a + \varepsilon)b$)

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

② 代入求解：用子方程替换父方程

$$S = a(\underbrace{aS + b}_A) + b = aaS + ab + b$$

$$S = (aa)^*(ab + b) \quad (\text{应用 Arden 定理})$$

③ 化简表达式: $S = (aa)^*(a + \varepsilon)b$ (因 $ab + b = (a + \varepsilon)b$)

例 2

求正则表达式 $(a+ab)^*$ 所描述的语言的正规文法。

① 写出方程：为每个非终结符建立方程

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

② 代入求解：用子方程替换父方程

$$S = a(\underbrace{aS + b}_A) + b = aaS + ab + b$$

$$S = (aa)^*(ab + b) \quad (\text{应用 Arden 定理})$$

③ 化简表达式： $S = (aa)^*(a + \varepsilon)b$ (因 $ab + b = (a + \varepsilon)b$)

验证方法：

- 用小字符串验证： $S \Rightarrow b$ (匹配 “b”), $S \Rightarrow aA \Rightarrow ab$ (匹配 “ab”)

福州大学计算机与大数据学院

18 / 86

① 写出方程：为每个非终结符建立方程

- $S \rightarrow aA \mid b \Rightarrow S = aA + b$
- $A \rightarrow aS \mid b \Rightarrow A = aS + b$

② 代入求解：用子方程替换父方程

$$S = a(\underbrace{aS + b}_A) + b = aaS + ab + b$$
$$S = (aa)^*(ab + b) \quad (\text{应用 Arden 定理})$$

③ 化简表达式： $S = (aa)^*(a + \varepsilon)b$ （因 $ab + b = (a + \varepsilon)b$ ）

验证方法：

- **检查生成语言：** $L(G) = \{w \mid w \text{ 以 } b \text{ 结尾}\}$

福州大学计算机与大数据学院

文法

$$S \rightarrow aS \mid b$$
$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

- 验证：

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

- 验证:
 - $S \Rightarrow b \rightarrow$ 匹配 b

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

- 验证:
 - $S \Rightarrow b \rightarrow$ 匹配 b
 - $S \Rightarrow aS \Rightarrow ab \rightarrow$ 匹配 ab

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

- 验证:
 - $S \Rightarrow b \rightarrow$ 匹配 b
 - $S \Rightarrow aS \Rightarrow ab \rightarrow$ 匹配 ab
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow aab \rightarrow$ 匹配 aab

文法

$$S \rightarrow aS \mid b$$

$$S \rightarrow a \mid b \quad (\text{等价于 } S \rightarrow aS \mid b)$$

- 方程: $S = aS + b$
- 应用 Arden 定理:

$$S = a^*b$$

- 验证:
 - $S \Rightarrow b \rightarrow$ 匹配 b
 - $S \Rightarrow aS \Rightarrow ab \rightarrow$ 匹配 ab
 - $S \Rightarrow aS \Rightarrow aaS \Rightarrow aab \rightarrow$ 匹配 aab
 - $L(G) = \{a^n b \mid n \geq 0\} = a^*b$

文法

$$\begin{aligned} S &\rightarrow aA \mid b \\ A &\rightarrow aS \mid b \end{aligned}$$

文法

$$S \rightarrow aA \mid b$$

$$A \rightarrow aS \mid b$$

- 方程组:

$$S = aA + b$$

$$A = aS + b$$

- 求解过程：

$$S = a(aS + b) + b = aaS + ab + b$$

$$S = (aa)^*(ab + b) = (aa)^*(a + \varepsilon)b$$

- 生成语言：

文法

$$A \rightarrow aS \mid b$$

- 方程组:

$$A = aS + b$$

- 求解过程：

$$S = (aa)^*(ab + b) = (aa)^*(a + \varepsilon)b$$

- 生成语言：

- b (直接匹配)

文法

$$A \rightarrow aS \mid b$$

- 方程组:

$$A = aS + b$$

- 求解过程：

$$S = (aa)^*(ab + b) = (aa)^*(a + \varepsilon)b$$

- 生成语言：

- b (直接匹配)
- ab ($S \rightarrow aA \rightarrow ab$)

文法

$$A \rightarrow aS \mid b$$

- $$S = (aa)^*(ab + b) = (aa)^*(a + \varepsilon)b$$

- aab ($S \rightarrow aA \rightarrow aaS \rightarrow aab$)

文法

$$A \rightarrow aS \mid b$$

- 方程组:

$$A = aS + b$$

- 求解过程：

$$S = (aa)^*(ab + b) = (aa)^*(a + \varepsilon)b$$

- 生成语言：

- b (直接匹配)
- ab ($S \rightarrow aA \rightarrow ab$)
- aab ($S \rightarrow aA \rightarrow aaS \rightarrow aab$)
- $L(G) = \{w \mid w \text{ 以 } b \text{ 结尾}\}$

A large, faint watermark of the Fuzhou University logo is centered on the page. The logo is circular, featuring the university's name in English, "FUZHOU UNIVERSITY", around the top and the Chinese characters "福州大学" around the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune". Below the main text, the year "1958" is visible, indicating the university's founding year. At the very bottom of the page, there is a small navigation bar with several icons for navigating through the presentation slides.

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ↺ 🔍 ↻

依赖的非终结符开始

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

依赖的非终结符开始
需拆分方程

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

依赖的非终结符开始
需拆分方程
 $a + \epsilon)b$ 等恒等式

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

依赖的非终结符开始
需拆分方程
 $a + \epsilon)b$ 等恒等式

- **关键技巧:**
 - **优先处理无循环的方程:** 从无依赖的非终结符开始
 - **避免 ε 问题:** 若 A 包含 ε , 需拆分方程
 - **化简表达式:** 利用 $ab + b = (a + \varepsilon)b$ 等恒等式
- **常见错误:**

依赖的非终结符开始
需拆分方程
 $a + \epsilon)b$ 等恒等式
 $a^* + b$ (漏掉 b 的闭包)

- 依赖的非终结符开始
需拆分方程
 $a + \epsilon)b$ 等恒等式
 $a^* + b$ (漏掉 b 的闭包)

依赖的非终结符开始
 需拆分方程
 $(a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)

- 依赖的非终结符开始
 需拆分方程
 $(a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)

依赖的非终结符开始
 需拆分方程
 $(a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 a^*b^* (混淆 $+$ 与 $*$)

- 依赖的非终结符开始
 需拆分方程
 $(a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 a^*b^* (混淆 $+$ 与 $*$)

依赖的非终结符开始
需拆分方程
 $a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)
 b, aab

- 依赖的非终结符开始
需拆分方程
 $a + \epsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)
 b, aab

依赖的非终结符开始
 需拆分方程
 $(a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)
 b, aab

- ① 用文法生成小字符串 (如 b, ab, aab)
- ② 用正则表达式匹配相同字符串

依赖的非终结符开始
需拆分方程
 $a + \varepsilon)b$ 等恒等式
 $a + b$ (漏掉 b 的闭包)
 b^* (混淆 $+$ 与 $*$)
 $b, aab)$
含所有以 b 结尾的字符串)

- b 结尾的字符串)**

自动机模型

- 自动机 (Automaton) 是一种计算模型



自动机模型

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态

算模型

ate) 转移 (Transition) 到另一个状态

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态
- 按照状态数量可以分为:

自动机模型

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态
- 按照状态数量可以分为：
 - 有限状态自动机 (Finite State Automaton)

算模型

ate) 转移 (Transition) 到另一个状态

Automaton)

e Automaton)

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态
- 按照状态数量可以分为：
 - 有限状态自动机 (Finite State Automaton)
 - 无限状态自动机 (Infinite State Automaton)

计算模型

State) 转移 (Transition) 到另一个状态

Automaton)

Automaton)

法在现实世界中实现，因此通常不在讨论范围

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态
- 按照状态数量可以分为：
 - 有限状态自动机 (Finite State Automaton)
 - 无限状态自动机 (Infinite State Automaton)
- 严格意义上的无限状态自动机无法在现实世界中实现，因此通常不在讨论范围

算模型

(state) 转移 (Transition) 到另一个状态

Automaton)

Automaton)

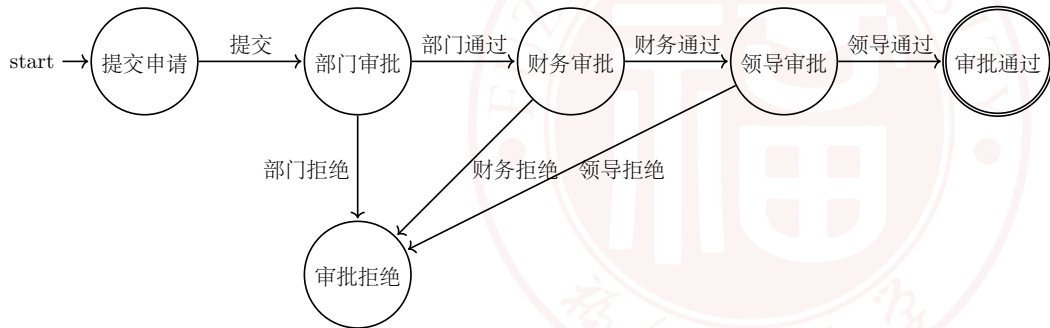
法在现实世界中实现，因此通常不在讨论范围

自动机及其推广而来的下推自动机 (Pushdown machine)

- 自动机 (Automaton) 是一种计算模型
- 它可以根据输入从一个状态 (State) 转移 (Transition) 到另一个状态
- 按照状态数量可以分为：
 - 有限状态自动机 (Finite State Automaton)
 - 无限状态自动机 (Infinite State Automaton)
- 严格意义上的无限状态自动机无法在现实世界中实现，因此通常不在讨论范围
- 通常计算机学科只讨论有限状态自动机及其推广而来的下推自动机 (Pushdown Automaton)、图灵机 (Turing Machine)

有限自动机

有限自动机示例



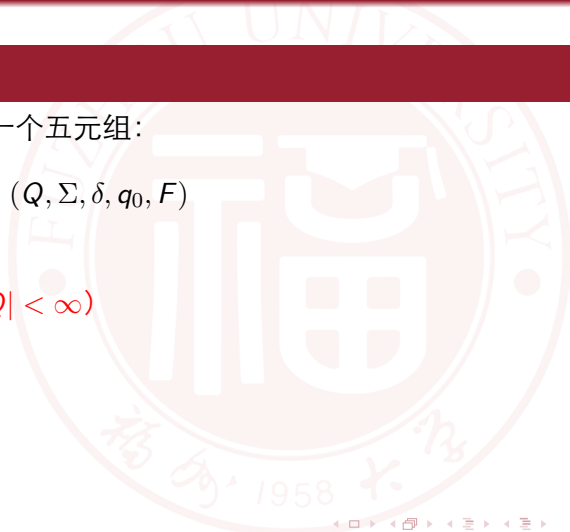
定义 (有限自动机)

有限自动机 (Finite Automaton) 是一个五元组:

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

其中：

- ① Q 是有限非空状态集合 ($0 < |Q| < \infty$)



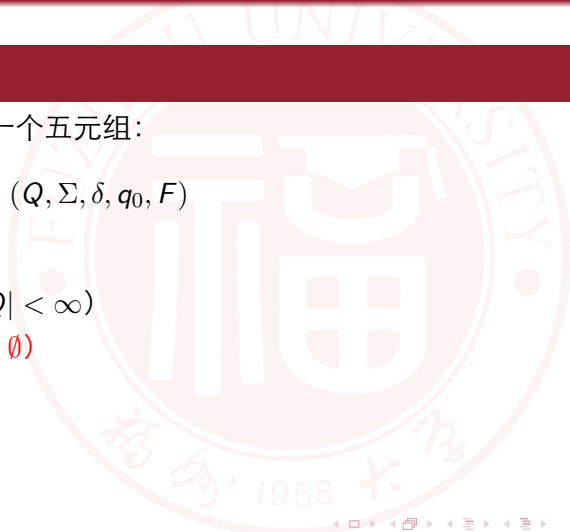
定义 (有限自动机)

有限自动机 (Finite Automaton) 是一个五元组:

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

其中：

- 1 Q 是有限非空状态集合 ($0 < |Q| < \infty$)
- 2 Σ 是有限非空输入字母表 ($\Sigma \neq \emptyset$)



定义 (有限自动机)

有限自动机 (Finite Automaton) 是一个五元组:

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

其中：

- ① Q 是有限非空状态集合 ($0 < |Q| < \infty$)
- ② Σ 是有限非空输入字母表 ($\Sigma \neq \emptyset$)
- ③ $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$ 是转移函数

器读的状态

状态 寄存器、程序计数器
输入 中断、指令、内存读
转移

接受态 可以继续执行指令的状态

身边的例子

- CPU:

状态 寄存器、程序计数器

输入 中断、指令、内存读

转移

接受态 可以继续执行指令的状态

- 自动售货机

状态 等待、投币中、选择商品、出货、找零

输入 投币行为、屏幕点击、按下按钮等

器
读
的状态
择商品、出货、找零
击、按下按钮等
状态转移

- | | |
|-----|-------------|
| 状态 | 寄存器、程序计数器 |
| 输入 | 中断、指令、内存读 |
| 转移 | |
| 接受态 | 可以继续执行指令的状态 |

- | | |
|----|-------------------|
| 状态 | 等待、投币中、选择商品、出货、找零 |
| 输入 | 投币行为、屏幕点击、按下按钮等 |

- 请设计自动柜员机 (ATM) 的状态转移

例

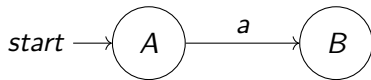
基于字符集 $\Sigma = \{a, b\}$ ，构建一个仅接受 $aaab$ 的 DFA

DFA 示例 1

例

基于字符集 $\Sigma = \{a, b\}$ ，构建一个仅接受 $aaab$ 的 DFA

解



DFA 示例 1

例

基于字符集 $\Sigma = \{a, b\}$ ，构建一个仅接受 $aaab$ 的 DFA

解

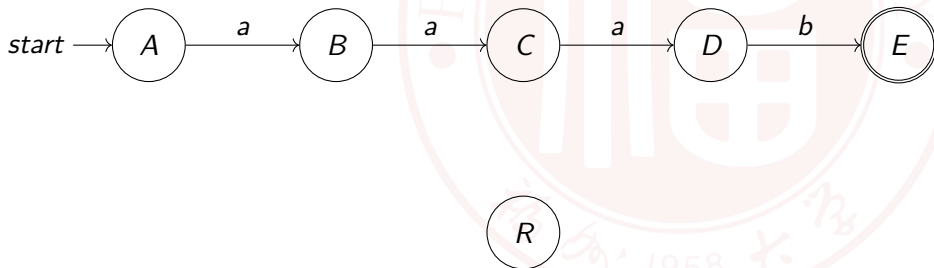


DFA 示例 1

例

基于字符集 $\Sigma = \{a, b\}$ ，构建一个仅接受 $aaab$ 的 DFA

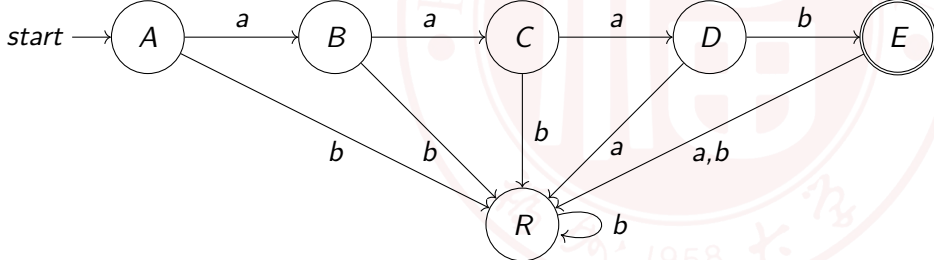
解



100

2

100



- 字符串“101”的 DFA

- 字符串“101”的 DFA
- 字符串“abca”的 DFA

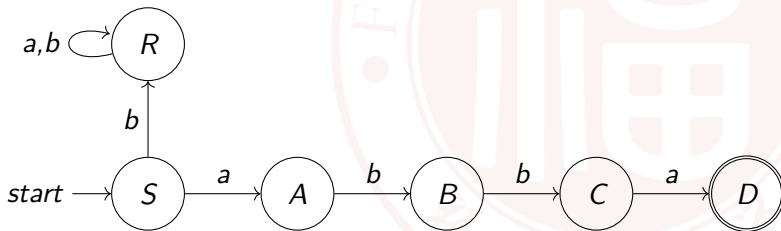
例

字符集 $\{a, b\}$ ，画出一个接受所有以 a 开头，以 bba 结尾的字符串。

例

字符集 $\{a, b\}$ ，画出一个接受所有以 a 开头，以 bba 结尾的字符串。

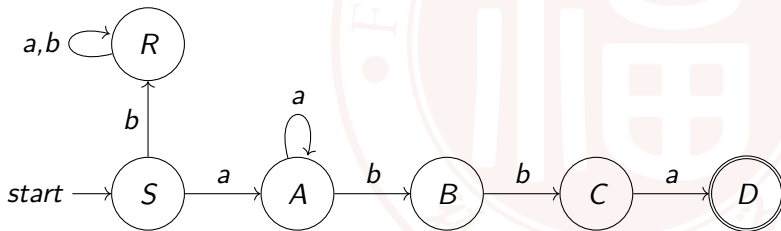
解



例

字符集 $\{a, b\}$ ，画出一个接受所有以 a 开头，以 bba 结尾的字符串。

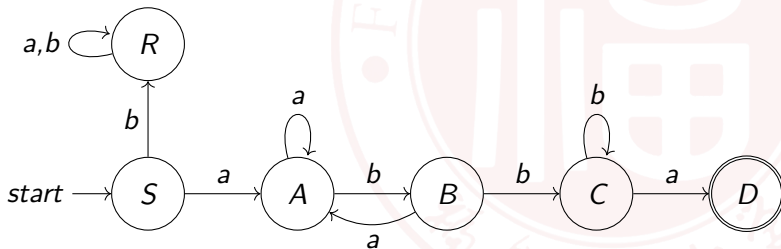
解



例

字符集 $\{a, b\}$ ，画出一个接受所有以 a 开头，以 bba 结尾的字符串。

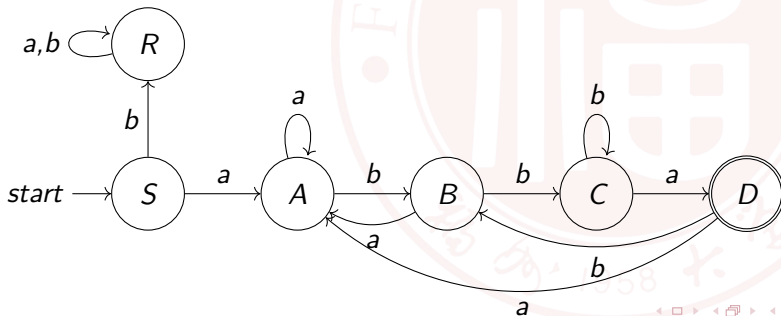
解



例

字符集 $\{a, b\}$ ，画出一个接受所有以 a 开头，以 bba 结尾的字符串。

解



DFA 练习 2

画出接受下列字符串的 DFA:

- 只包含 0 或者 1 的 01 字符串



DFA 练习 2

画出接受下列字符串的 DFA:

- 只包含 0 或者 1 的 01 字符串
- 以 “01” 结尾的字符串



- 只包含 0 或者 1 的 01 字符串
- 以 “01” 结尾的字符串
- 以 “01” 或者 “10” 结尾的任意字符串

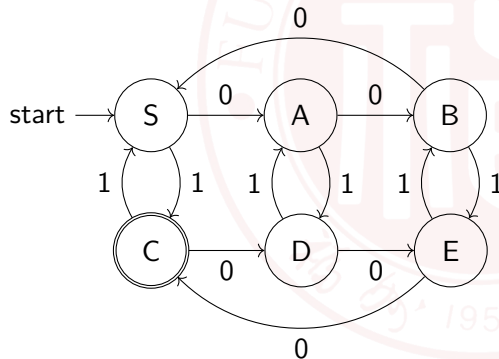
- 只包含 0 或者 1 的 01 字符串
- 以 “01” 结尾的字符串
- 以 “01” 或者 “10” 结尾的任意字符串
- 开始、结尾字符不同的 01 字符串

例

字符集 $\{0, 1\}$ ，画出一个接受所有“0”的数量恰好能被 3 整除且有奇数个“1”的字符串。

例

字符集 $\{0, 1\}$ ，画出一个接受所有“0”的数量恰好能被 3 整除且有奇数个“1”的字符串。



例

字符集 $\{0, 1\}$ ，设计一个能够接受有至少一个长度为 5 的子串重复出现过的字符串。

匹配

- 
- 匹配

——匹配

- 自动机和正则表达式的机制都是——匹配
- 正则表达式可以有多个选择

——匹配

- 自动机和正则表达式的机制都是——匹配
- 正则表达式可以有多个选择
- 确定性自动机只有一个选择

——匹配

- 自动机和正则表达式的机制都是——匹配
- 正则表达式可以有多条选择
- 确定性自动机只有一个选择
- 存在一个从多到一的鸿沟

——匹配

- 自动机和正则表达式的机制都是——匹配
- 正则表达式可以有多条选择
- 确定性自动机只有一个选择
- 存在一个从多到一的鸿沟
- 用非确定性来填补

第二种定义

定义 (确定性有限自动机)

确定性有限自动机 (Deterministic Finite Automaton) 是一个五元组:

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

其中:

- ① Q 是有限非空状态集合 ($0 < |Q| < \infty$)
- ② Σ 是有限非空输入字母表 ($\Sigma \neq \emptyset$)
- ③ $\delta: Q \times \Sigma \rightarrow Q$ 是转移函数
- ④ $q_0 \in Q$ 是初始状态
- ⑤ $F \subseteq Q$ 是接受状态集合

第二种定义

定义 (非确定性有限自动机)

非确定性有限自动机 (Nondeterministic Finite Automaton) 是一个五元组:

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

其中:

- ① Q 是有限非空状态集合 ($0 < |Q| < \infty$)
- ② Σ 是有限非空输入字母表 ($\Sigma \neq \emptyset$)
- ③ $\delta: Q \times \{\epsilon\} \cup \Sigma \rightarrow Q$ 是转移函数
- ④ $q_0 \in Q$ 是初始状态
- ⑤ $F \subseteq Q$ 是接受状态集合

用 ϵ -NFA 来构建。

“更构造”，状态集合法是为了“简洁定义”。

论的视角差异。

- 无 ε 的 NFA 理论干净清晰。
- 实际编译器/正则工具中，几乎都用 ε -NFA 来构建。
- 空转移法 (ε -NFA) 是为了“方便构造”，状态集合法是为了“简洁定义”。
- 两者能力相同，只是工程 vs 理论的视角差异。

例

构造一个 NFA 使其能够接受所有长度不少于 3 且倒数第 3 个字符是 a 的 ab 字符串。

例

构造一个 NFA 使其能够接受所有长度不少于 3 且倒数第 3 个字符是 a 的 ab 字符串。



例

构造一个 NFA 使其能够接受所有长度不少于 3 且倒数第 3 个字符是 a 的 ab 字符串。



如果用 DFA 实现要怎么做呢?

NFA 的力量 2

例

构造一个 NFA 使其能够接受所有包含子串 `abbaa` 的 `ab` 字符串。

含子串 abbaa 的 ab 字符串。

例

构造一个 NFA 使其能够接受所有包含子串 `abbaa` 的 `ab` 字符串。

如果用 DFA 实现要怎么做呢?

NFA 小结

问题	NFA 状态数	DFA 状态数	谁更方便
匹配正则表达式 a^*b^*	3~4	3~4	差不多
倒数第 n 个字符是 a	$O(n)$	2^n	NFA
包含子串 α	子串长度+2	子串长度+x	NFA 转移简单, DFA 复杂

目录



① 正则文法与正则表达式

② 有限自动机

确定性有限自动机

非确定性有限自动机

其他理论模型与复杂度类

子集构造法

正则表达式与 NFA 的等价性

最小化 DFA

③ 词法分析器生成器

④ 小结与作业

其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$



其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$



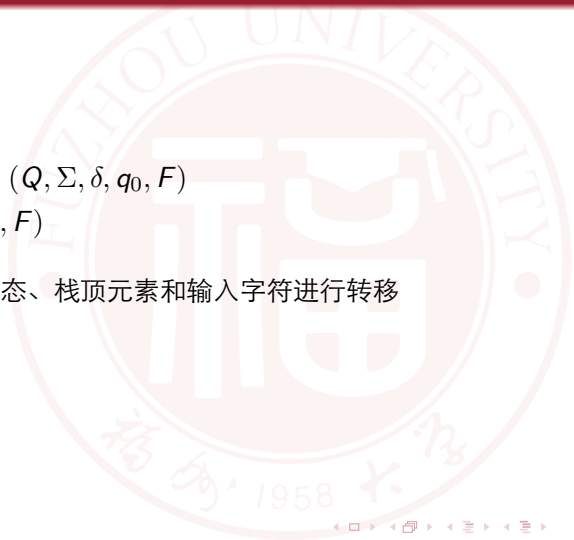
其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta : Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \Gamma^*$



其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta: Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \Gamma^*$
 - 需要维护一个栈，根据当前状态、栈顶元素和输入字符进行转移

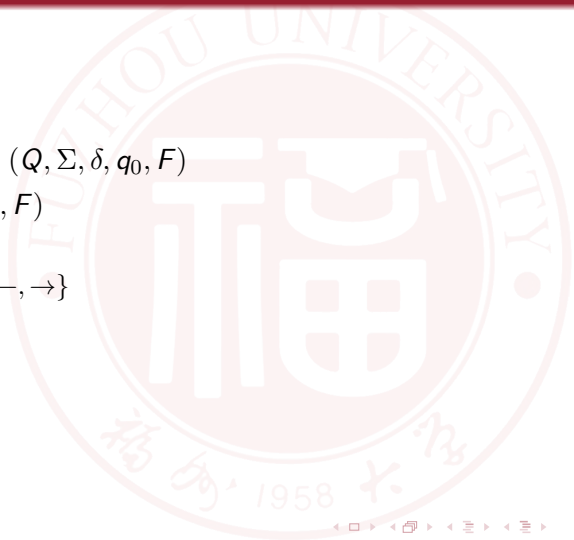


其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta: Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \Gamma^*$
 - 需要维护一个栈，根据当前状态、栈顶元素和输入字符进行转移
 - 非确定性与确定性下推自动机无法等价

其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
- 图灵机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta: Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \{\Gamma, \leftarrow, \rightarrow\}$



其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
- 图灵机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta: Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \{\Gamma, \leftarrow, \rightarrow\}$
 - 需要维护一个纸带和操作头, 根据当前状态、读头字符和输入字符进行转移

其他模型

- 非确定性有限状态自动机: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
- 下推自动机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
- 图灵机: $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, F)$
 - $\delta: Q \times \Gamma \times \Sigma \rightarrow \mathcal{P}(Q) \times \{\Gamma, \leftarrow, \rightarrow\}$
 - 需要维护一个纸带和操作头, 根据当前状态、读头字符和输入字符进行转移
 - 确定性图灵机和非确定性图灵机分别代表了 2 种不同的复杂度类

复杂度类

P-问题 所有可以由一个确定型图灵机在多项式表达的时间内解决的问题



复杂度类

P-问题 所有可以由一个确定型图灵机在多项式表达的时间内解决的问题

NP-问题 可以在非确定型图灵机上在多项式时间内找出解的问题

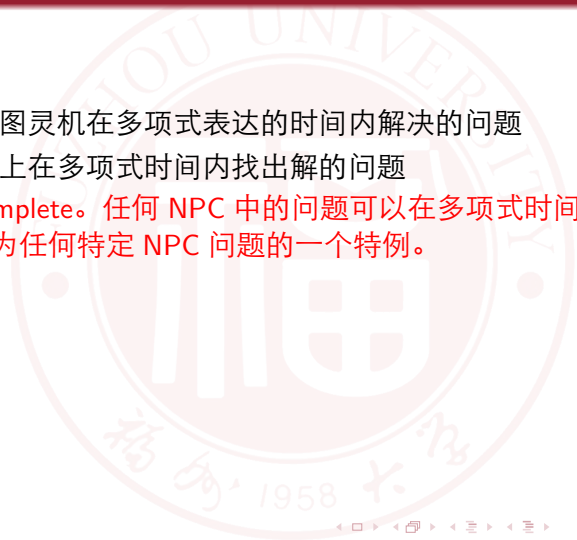


复杂度类

P-问题 所有可以由一个确定型图灵机在多项式表达的时间内解决的问题

NP-问题 可以在非确定型图灵机上在多项式时间内找出解的问题

NPC 问题 即 NP-Complete。任何 NPC 中的问题可以在多项式时间内变换成为任何特定 NPC 问题的一个特例。



复杂度类

P-问题 所有可以由一个确定型图灵机在多项式表达的时间内解决的问题

NP-问题 可以在非确定型图灵机上在多项式时间内找出解的问题

NPC 问题 即 NP-Complete。任何 NPC 中的问题可以在多项式时间内变换成为任何特定 NPC 问题的一个特例。

NP-hard 问题 至少与 NP 完全问题一样难。哪怕知道了一个解，也不一定能够在多项式时间内验证。

复杂度类

P-问题 所有可以由一个确定型图灵机在多项式表达的时间内解决的问题

NP-问题 可以在非确定型图灵机上在多项式时间内找出解的问题

NPC 问题 即 NP-Complete。任何 NPC 中的问题可以在多项式时间内变换成为任何特定 NPC 问题的一个特例。

NP-hard 问题 至少与 NP 完全问题一样难。哪怕知道了一个解，也不一定能够在多项式时间内验证。

PSPACE 问题 所有可以在确定性图灵机上用多项式空间（内存）解决的判定问题组成的集合。

目录



① 正则文法与正则表达式

② 有限自动机

确定性有限自动机

非确定性有限自动机

其他理论模型与复杂度类

子集构造法

正则表达式与 NFA 的等价性

最小化 DFA

③ 词法分析器生成器

④ 小结与作业

NFA→DFA

原因

- 现代计算机是确定性的，无法处理非确定性

NFA→DFA

原因

- 现代计算机是确定性的，无法处理非确定性
- 确定性算法结果确定、处理高效

NFA→DFA

原因

- 现代计算机是确定性的，无法处理非确定性
- 确定性算法结果确定、处理高效

核心思想

- 用“空间”换“时间”

NFA→DFA

原因

- 现代计算机是确定性的，无法处理非确定性
- 确定性算法结果确定、处理高效

核心思想

- 用“空间”换“时间”
- 将可能性作为一种状态

表示所有可能状态的集合状态——子集构造法

- ① 初始状态：计算 NFA 初始状态的 ε -闭包（所有通过空转移可达的状态）
- ② 状态转移：对每个当前状态子集 S 和输入符号 a ，计算：

$$\delta_{\text{DFA}}(S, a) = \varepsilon\text{-closure} \left(\bigcup_{q \in S} \delta_{\text{NFA}}(q, a) \right)$$

- ③ 迭代：将新状态加入队列，重复直到无新状态
- ④ 接受状态：包含 NFA 接受状态的子集

- 
- A large, faint watermark of the Fuzhou University logo is centered on the page. The logo is circular, featuring the university's name in English, "FUZHOU UNIVERSITY", around the top and the Chinese characters "福州大学" around the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune". Below the character "福" is the year "1958". The entire logo is rendered in a light gray color, serving as a background element.

示例：NFA $\rightarrow$ DFA

NFA 定义

- 状态: $Q = \{A, B\}$
- 输入: $\Sigma = \{0, 1\}$
- 转移: $A \xrightarrow{0} B, A \xrightarrow{1} A, B \xrightarrow{1} B$
- 接受状态: $F = \{B\}$

转换过程

- $\varepsilon\text{-closure}(A) = \{A\}$ (初始状态)
- $\{A\} \xrightarrow{0} \{B\}, \{A\} \xrightarrow{1} \{A\}$
- $\{B\} \xrightarrow{0} \emptyset, \{B\} \xrightarrow{1} \{B\}$
- $\emptyset \xrightarrow{0/1} \emptyset$ (死状态)

示例：NFA $\rightarrow$ DFA

NFA 定义

- 状态: $Q = \{A, B\}$
- 输入: $\Sigma = \{0, 1\}$
- 转移: $A \xrightarrow{0} B, A \xrightarrow{1} A, B \xrightarrow{1} B$
- 接受状态: $F = \{B\}$

等价 DFA

- 状态: $\{A\}, \{B\}, \emptyset$
- 初始: $\{A\}$
- 接受: $\{B\}$

转换过程

- $\varepsilon\text{-closure}(A) = \{A\}$ (初始状态)
- $\{A\} \xrightarrow{0} \{B\}, \{A\} \xrightarrow{1} \{A\}$
- $\{B\} \xrightarrow{0} \emptyset, \{B\} \xrightarrow{1} \{B\}$
- $\emptyset \xrightarrow{0/1} \emptyset$ (死状态)

数 $= 2^{|Q_{\text{NFA}}|}$ (如 NFA 有 5 状态, DFA 最多 32 状态)

正则表达式, DFA 用于高效匹配 (输入长度), 无需回溯

由正则表达式生成的 NFA 通常状态少, 可能将形式化理论转化为实用工具

- 数 $= 2^{|Q_{\text{NFA}}|}$ (如 NFA 有 5 状态, DFA 最多 32 状态)
- 正则表达式, DFA 用于高效匹配 (输入长度), 无需回溯
- 由正则表达式生成的 NFA 通常状态少, 可能更优
- 理论转化为实用工具

目录



① 正则文法与正则表达式

② 有限自动机

确定性有限自动机

非确定性有限自动机

其他理论模型与复杂度类

子集构造法

正则表达式与 NFA 的等价性

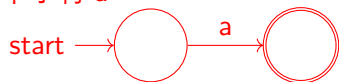
最小化 DFA

③ 词法分析器生成器

④ 小结与作业

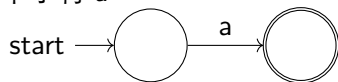
从正则表达式到 NFA

- 单字符 a:

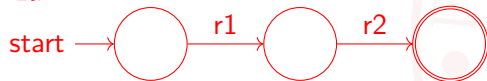


从正则表达式到 NFA

- 单字符 a :

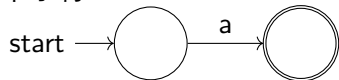


- 连接 $r_1 r_2$:

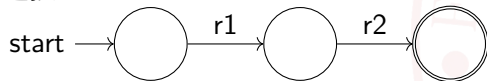


从正则表达式到 NFA

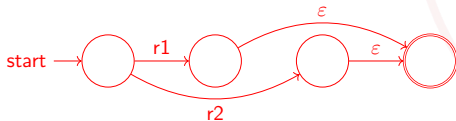
- 单字符 a :



- 连接 r_1r_2 :

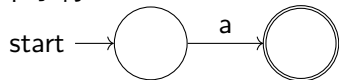


- 选择 $r_1|r_2$:

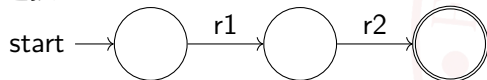


从正则表达式到 NFA

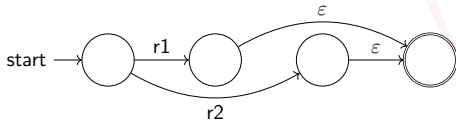
- 单字符 a :



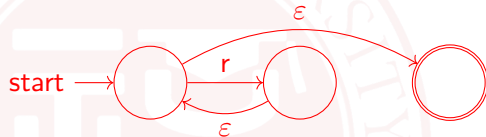
- 连接 $r_1 r_2$:



- 选择 $r_1 | r_2$:



- r^* :



-
- ```

graph LR
 start((start)) -- r1 --> S2((()))
 start -- r2 --> S3((()))
 S2 -- ε --> S3
 S3 -- ε --> S4((()))

```

- 
- ```

graph LR
    start(( )) -- start --> S1(( ))
    S1 -- r --> S2(( ))
    S2 -- ε --> S3((( )))
    S3 -- ε --> S1
  
```

RE→NFA 示例

例

画出下列正则表达式对应的 NFA

$0^*1^*(01^+)^*0?$

解

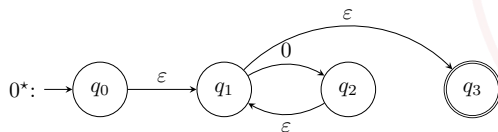
RE→NFA 示例

例

画出下列正则表达式对应的 NFA

$$0^*1^*(01^+)^*0?$$

解



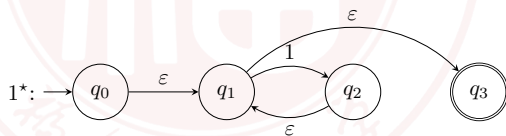
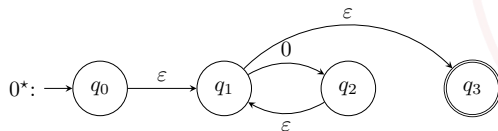
RE $\rightarrow$ NFA 示例

例

画出下列正则表达式对应的 NFA

$$0^*1^*(01^+)^*0?$$

解



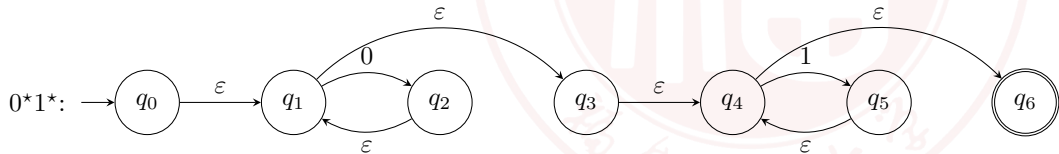
RE → NFA 示例

例

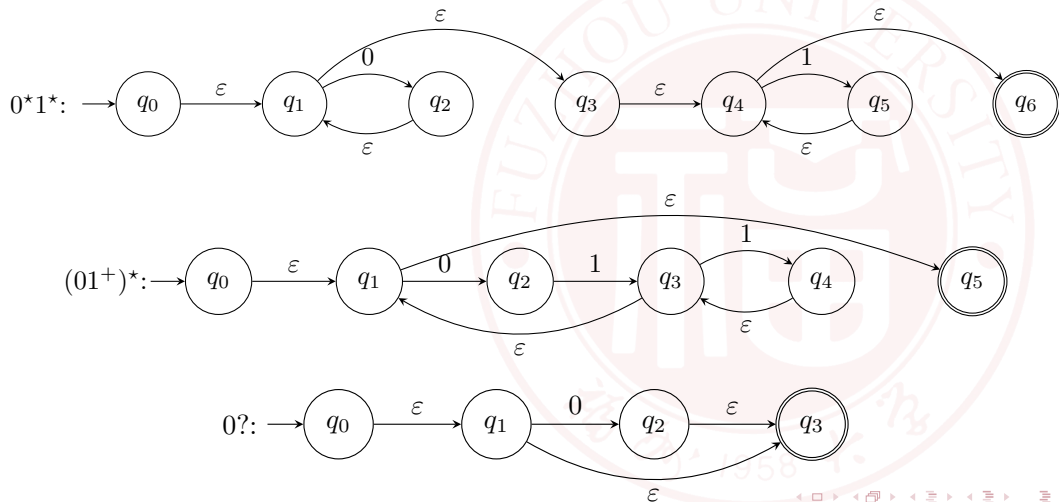
画出下列正则表达式对应的 NFA

$$0^*1^*(01^+)^*0?$$

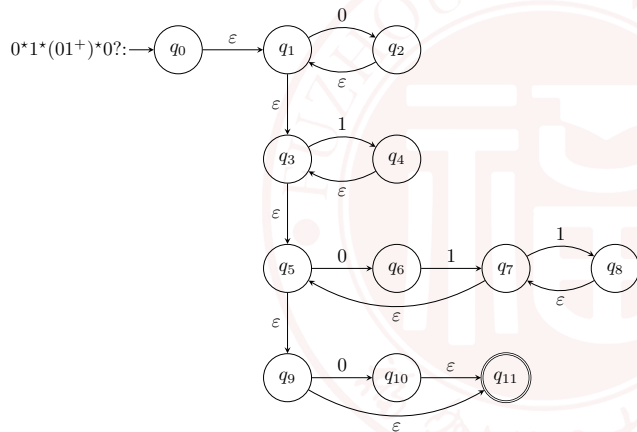
解



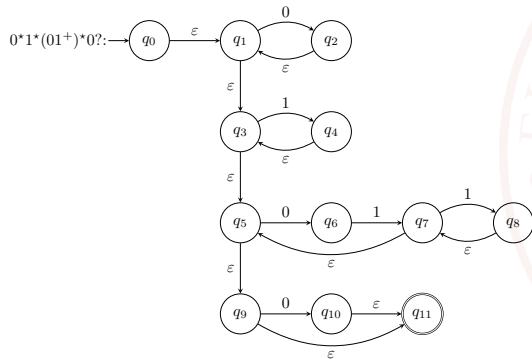
RE→NFA 示例 (续 1)



RE→NFA 示例 (续 2)



从 NFA 到 DFA



状态	0	1	ε
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

NFA→DFA: 计算 ϵ -闭包

状态	0	1	ϵ
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			
1	2		
2			
3		4	
4			
5	6		
6		7	
7		8	
8			
9	10		
10			

NFA→DFA: 计算 ϵ -闭包

状态	0	1	ϵ
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		
2			
3		4	
4			
5	6		
6		7	
7		8	
8			
9	10		
10			

NFA→DFA: 计算 ϵ -闭包

状态	0	1	ϵ
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			
3		4	
4			
5	6		
6		7	
7		8	
8			
9	10		
10			

NFA→DFA: 计算 ϵ -闭包

状态	0	1	ϵ
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	
4			
5	6		
6		7	
7		8	
8			
9	10		
10			

NFA→DFA: 计算 ϵ -闭包

状态	0	1	ϵ
0			1
1	2		3
2			1
3		4	5
4			3
5	6		9
6		7	
7		8	5
8			7
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			
5	6		
6		7	
7		8	
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		
6		7	
7		8	
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		
6		7	
7		8	
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			
9	10		
10			

状态	0	1	ε -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		
10			

状态	0	1	ε -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			

状态	0	1	ε -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			11

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			11

NFA→DFA：可能状态集合计算

状态	0	1	ϵ -转移可达状态
0			1, 3, 5, 9, 11
1	2		3, 5, 9, 11
2			1, 3, 5, 9, 11
3		4	5, 9, 11
4			3, 5, 9, 11
5	6		9, 11
6		7	
7		8	5, 9, 11
8			7, 5, 9, 11
9	10		11
10			11

编号	可状态集合	0	1
A	{0, 1, 3, 5, 9, 11}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11}
B	{2, 1, 3, 5, 9, 11, 6, 10}	{2, 1, 3, 5, 9, 11, 6, 10}	{4, 3, 5, 9, 11, 7}
C	{4, 3, 5, 9, 11}	{6, 10, 11}	{4, 3, 5, 9, 11}
D	{4, 3, 5, 9, 11, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
E	{6, 10, 11}	{}	{7, 5, 9, 11}
F	{4, 3, 5, 9, 11, 8, 7}	{6, 10, 11}	{4, 3, 5, 9, 11, 8, 7}
G	{7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
H	{8, 7, 5, 9, 11}	{6, 10, 11}	{8, 7, 5, 9, 11}
J	{}	{}	{}

目录



① 正则文法与正则表达式

② 有限自动机

- 确定性有限自动机
- 非确定性有限自动机
- 其他理论模型与复杂度类
- 子集构造法
- 正则表达式与 NFA 的等价性
- 最小化 DFA

③ 词法分析器生成器

④ 小结与作业

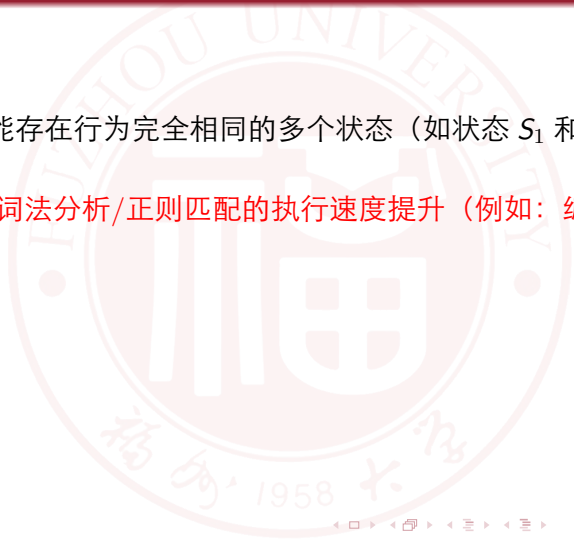
为什么需要最小化 DFA?

- 消除冗余状态：原始 DFA 中可能存在行为完全相同的多个状态（如状态 S_1 和 S_2 对所有输入的转移路径相同）



为什么需要最小化 DFA?

- 消除冗余状态：原始 DFA 中可能存在行为完全相同的多个状态（如状态 S_1 和 S_2 对所有输入的转移路径相同）
- 提升计算效率：状态数减少 → 词法分析/正则匹配的执行速度提升（例如：编译器中状态转移表更小）



为什么需要最小化 DFA?

- 消除冗余状态：原始 DFA 中可能存在行为完全相同的多个状态（如状态 S_1 和 S_2 对所有输入的转移路径相同）
- 提升计算效率：状态数减少 $\rightarrow$ 词法分析/正则匹配的执行速度提升（例如：编译器中状态转移表更小）
- 标准化表示：最小化后的 DFA 是唯一确定的，便于理论分析与系统间比较

为什么需要最小化 DFA?

- 消除冗余状态：原始 DFA 中可能存在行为完全相同的多个状态（如状态 S_1 和 S_2 对所有输入的转移路径相同）
- 提升计算效率：状态数减少 $\rightarrow$ 词法分析/正则匹配的执行速度提升（例如：编译器中状态转移表更小）
- 标准化表示：最小化后的 DFA 是唯一确定的，便于理论分析与系统间比较

关键结论

最小化 DFA 是等价类划分的典型应用，确保状态数达到理论下限

“区分状态”实现

集合 F 和非接受态集 $Q \setminus F$

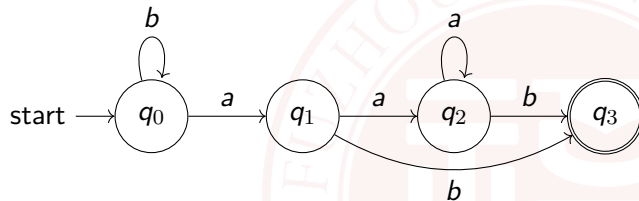
查输入符号是否使所有状态转移到同一组。若转移

止（达到稳定状态）

- “区分状态”实现
- 集合 F 和非接受态集 $Q \setminus F$
- 查输入符号是否使所有状态转移到同一组。若转移
- 终止（达到稳定状态）

算法复杂度

Hopcroft 算法: $O(|\Sigma| \cdot |Q| \log |Q|)$

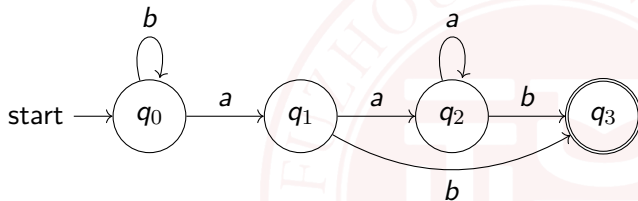


```

graph LR
    start(( )) --> q1((q1))
    q1 -- a --> q2((q2))
    q2 -- a --> q2
    q2 -- b --> q3(((q3)))
    q1 -- b --> q3
  
```

$q_0, q_1, q_2\}$

① 初始分组: $F = \{q_3\}$, $Q \setminus F = \{q_0, q_1, q_2\}$



```

graph LR
    start((start)) --> q0((q0))
    q0 -- b --> q0
    q0 -- a --> q1((q1))
    q1 -- a --> q2((q2))
    q1 -- b --> q3(((q3)))
    q2 -- a --> q2
    q2 -- b --> q3
    style start fill:none,stroke:none
    style q3 stroke-width:4px
  
```

最小化步骤：

- ① 初始分组： $F = \{q_3\}$, $Q \setminus F = \{q_0, q_1, q_2\}$
- ② q_1 和 q_2 在 a 输入下均转移到 $\{q_2\}$ （同组）， b 输入均到 $\{q_3\}$ （终态），而 q_0 在 b 输入下没有转移到终态 $\rightarrow$ 分裂为 $\{q_0\}$ 和 $\{q_1, q_2\}$ ，最终将 q_1, q_2 合并为 q_{12}
- ③ 最终状态： $\{q_0\}, \{q_{12}\}, \{q_3\}$

③ 最终状态: $\{q_0\}, \{q_{12}\}, \{q_3\}$

原始 DFA 状态转移表:

状态	a	b	类型
q_0	q_1	q_0	非终态
q_1	q_2	q_3	终态
q_2	q_2	q_3	终态
q_3	q_3	q_3	非终态

迭代划分过程：

① 初始划分： $\{q_1, q_2\}, \{q_0, q_3\}$

迭代划分过程:

状态	a	b	类型
q_0	q_1	q_0	非终态
q_1	q_2	q_3	终态
q_2	q_2	q_3	终态
q_3	q_3	q_3	非终态

① 初始划分: $\{q_1, q_2\}, \{q_0, q_3\}$

初拾捌日，（6月30日）

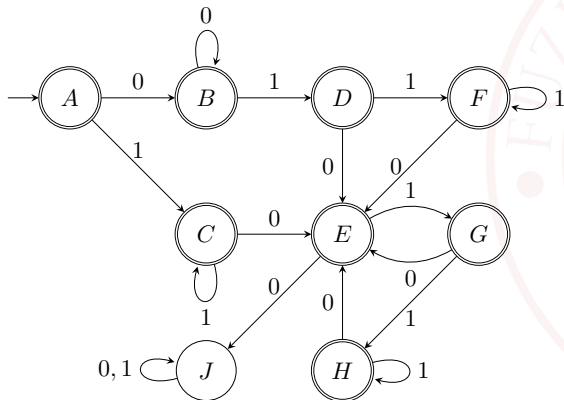
状态	a	b	类型
q_1	q_1	q_2	初始态
q_2	q_1	q_3	接受态

初拾捌日，（ ）（ ）（ ）（ ）

- | 状态 | a | b | 类型 |
|-------|-------|-------|-----|
| q_1 | q_1 | q_2 | 初始态 |
| q_2 | q_1 | q_3 | 接受态 |

挑战题

最小化下面的 DFA:



状态集合: $\{A, B, C, D, E, F, G, H, J\}$

初始状态: A

接受状态: $\{A, B, C, D, E, F, G, H\}$

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

最小化过程：等价类划分

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

最小化过程：等价类划分

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组

① 第1轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

最小化过程：等价类划分

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 = \{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内
- $C, D, F, G, H \xrightarrow{0} \{E\}$ 组

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

① 第1轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内
- $C, D, F, G, H \xrightarrow{0} \{E\}$ 组
- $P_2 = \{\{A, B\}, \{C, D, F, G, H\}, \{E\}, \{J\}\}$

最小化过程：等价类划分

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内
- $C, D, F, G, H \xrightarrow{0} \{E\}$ 组
- $P_2 = \{\{A, B\}, \{C, D, F, G, H\}, \{E\}, \{J\}\}$

④ 第4轮划分 (检查输入1的转移):

最小化过程：等价类划分

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内
- $C, D, F, G, H \xrightarrow{0} \{E\}$ 组
- $P_2 = \{\{A, B\}, \{C, D, F, G, H\}, \{E\}, \{J\}\}$

④ 第4轮划分 (检查输入1的转移):

- $\{A, B\}: A \xrightarrow{1} C, B \xrightarrow{1} D$ (同组) ✓

状态转移表:

状态	0	1	类型
A	B	C	终态
B	B	D	终态
C	E	C	终态
D	E	F	终态
E	J	G	终态
F	E	F	终态
G	E	H	终态
H	E	H	终态
J	J	J	非终态

① 第 1 轮划分 (按终态/非终态):

- $P_0 = \{\{A, B, C, D, E, F, G, H\}, \{J\}\}$

② 第2轮划分 (检查输入0的转移):

- $E \xrightarrow{0} J$ (非终态组)
- 其他状态 $\xrightarrow{0}$ 终态组
- $P_1 =$
 $\{\{A, B, C, D, F, G, H\}, \{E\}, \{J\}\}$

③ 第3轮划分 (检查输入0的转移):

- $A, B \xrightarrow{0} \{A, B, C, D, F, G, H\}$ 组内
- $C, D, F, G, H \xrightarrow{0} \{E\}$ 组
- $P_2 = \{\{A, B\}, \{C, D, F, G, H\}, \{E\}, \{J\}\}$

④ 第4轮划分 (检查输入1的转移):

- $\{A, B\}: A \xrightarrow{1} C, B \xrightarrow{1} D$ (同组) ✓
- $\{C, D, F, G, H\}$ 需进一步验证

深入分析 $\{C, D, F, G, H\}$ 组:

状态	输入 0	输入 1
C	E	C
D	E	F
F	E	F
G	E	H
H	E	H



最终划分结果

等价类:

$$P_{final} = \{\{A, B\}, \{C, D, F, G, H\}, \{E\}, \{J\}\}$$

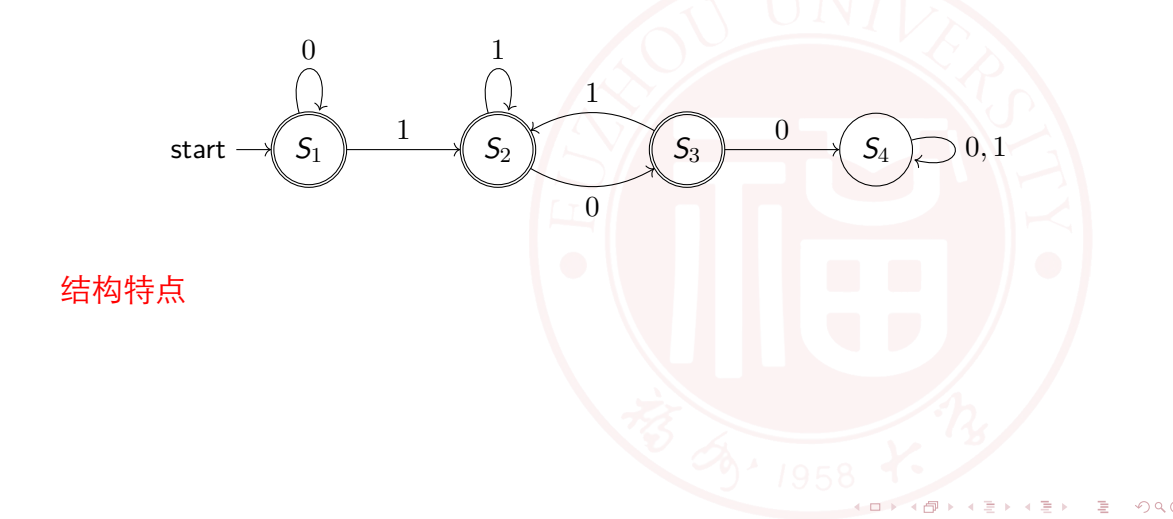
可合并的状态:

- ① $A \equiv B \rightarrow S_1$
- ② $C \equiv D \equiv F \equiv G \equiv H \rightarrow S_2$
- ③ $E \rightarrow S_3$ (独立)
- ④ $J \rightarrow S_4$ (独立)

最小化后状态: $9 \rightarrow 4$ 个状态

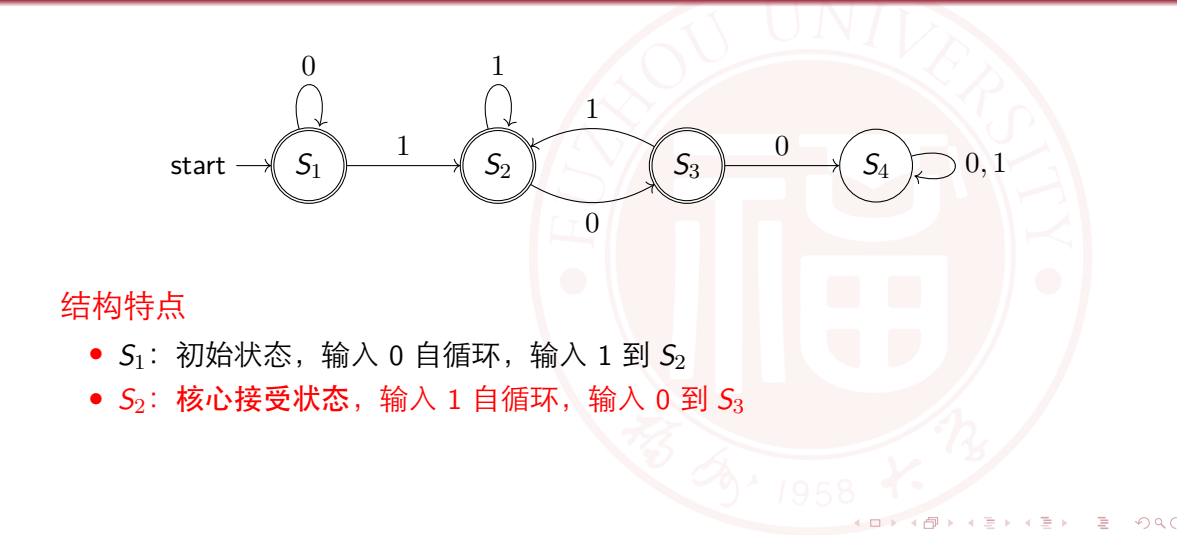
优化效果

$$\text{状态减少率} = \frac{9-4}{9} \times 100\% = 55.6\%$$



住住住

6. 初始状态 给 1, 0 白纸IT 给 1, 1 到 6

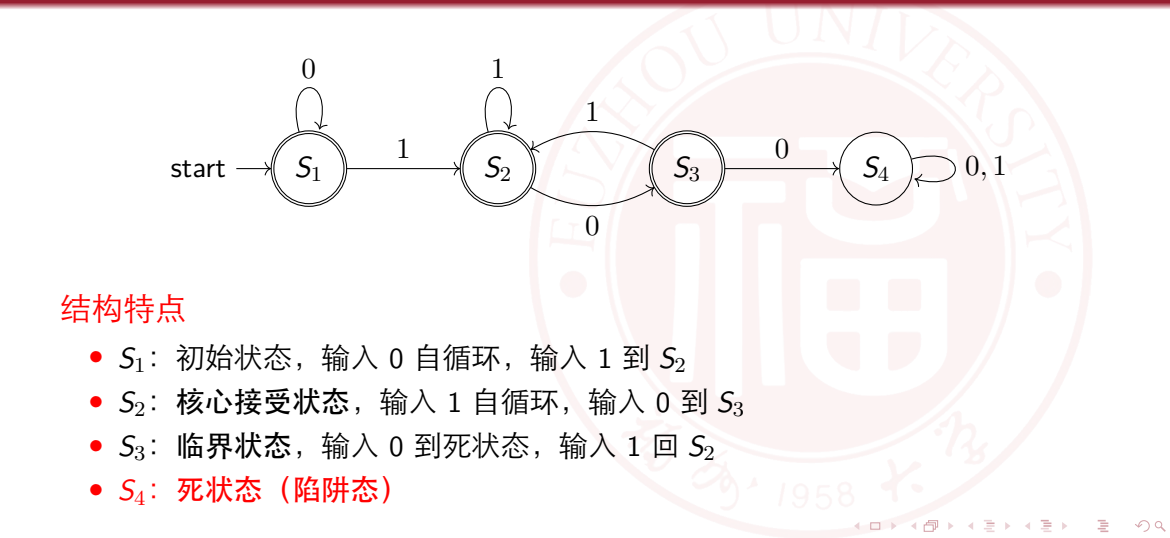


结构特点

- S_1 : 初始状态, 输入 0 自循环, 输入 1 到 S_2
- S_2 : 核心接受状态, 输入 1 自循环, 输入 0 到 S_3

住住住

6. 初始状态 给 1, 6 白纸片 给 1, 1 到 6



结构特点

- S_1 : 初始状态, 输入 0 自循环, 输入 1 到 S_2
- S_2 : 核心接受状态, 输入 1 自循环, 输入 0 到 S_3
- S_3 : 临界状态, 输入 0 到死状态, 输入 1 回 S_2
- S_4 : 死状态 (陷阱态)

① 初始状态为开始符号

① 初始状态为开始符号

生成 ε 的产生式

- ① 初始状态为开始符号
- ② 如果本身为接受态，则增加一个生成 ϵ 的产生式

生成 ε 的产生式
且 T 不是死状态，则增加一个 $S \rightarrow \sigma T$ 的产生式

- ① 初始状态为开始符号
- ② 如果本身为接受态，则增加一个生成 ϵ 的产生式
- ③ 对于任意一个转移 $\delta(S, \sigma) = T$ ，且 T 不是死状态，则增加一个 $S \rightarrow \sigma T$ 的产生式

- ① 正则文法与正则表达式
- ② 有限自动机
- ③ 词法分析器生成器
 - Flex
 - ANTLR
- ④ 小结与作业

- ① 正则文法与正则表达式
- ② 有限自动机
- ③ 词法分析器生成器
 - Flex
 - ANTLR
- ④ 小结与作业

的 GNU 实现，专用于生成词法分析器

驱动

```
)  
lexer lex.yy.c)  
  
JM: %s", yytext); }  
D: %s", yytext); }
```

- ```
[0-9]+ { printf("NUM: %s", yytext); }
[a-zA-Z]+ { printf("ID: %s", yytext); }
```

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

## ANTLR: 通用语言工具链

- **定位：**支持词法分析+语法分析的完整编译器工具链
- **核心优势：**
  - 基于 LL(k) 解析算法，支持复杂语法
  - 自动生成带 AST（抽象语法树）的解析器
  - 多语言后端（Java/C#/Python 等）
  - 内置词法分析器生成能力
- **工作流程：**
  - ① 编写 Grammar.g4（语法定义）
  - ② 运行 ANTLR 工具生成解析器
  - ③ 集成到目标语言项目
- **示例语法：**

```
grammar Expr;
expr: term ('+' term)*;
term: INT;
```

|   |                     |
|---|---------------------|
|   | <b>ANTLR</b>        |
|   | 词法+语法分析             |
|   | EBNF 语法             |
|   | 多语言 (Java/Python 等) |
|   | ✓                   |
|   | 高                   |
| 斤 | 完整编译器构建             |

| 特性     | Flex   | ANTLR              |
|--------|--------|--------------------|
| 定位     | 仅词法分析  | 词法+语法分析            |
| 输入格式   | 正则表达式  | EBNF 语法            |
| 输出语言   | C 代码   | 多语言（Java/Python 等） |
| AST 支持 | ×      | ✓                  |
| 学习曲线   | 低      | 高                  |
| 典型场景   | 快速词法分析 | 完整编译器构建            |



A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc. The year "1958" is positioned at the bottom center of the circle. In the middle of the logo is a large, stylized Chinese character "福" (Fú), which is a common symbol for good fortune. The entire watermark is rendered in a light gray or off-white color, serving as a background element.

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

- 正则表达式  $\leftrightarrow$  正则文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

- ## ● 关键算法总结:

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

表达式构造带  $\varepsilon$  边的 NFA。

- 正则表达式  $\leftrightarrow$  正则文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

- ### ① Thompson 构造法：从正则表达式构造带 $\epsilon$ 边的 NFA。

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

表达式构造带  $\varepsilon$  边的 NFA。

**Function):** 通过  $\varepsilon$ -closure 和 *move* 操作将 NFA 确定化

- ① **Thompson 构造法**: 从正则表达式构造带  $\epsilon$  边的 NFA。
- ② **子集构造法 (Subset Construction)**: 通过  $\epsilon$ -closure 和 *move* 操作将 NFA 确定化。

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

表达式构造带  $\varepsilon$  边的 NFA。

**化简 (Minimization)**: 通过  $\varepsilon$ -closure 和 *move* 操作将 NFA 确定化并消除冗余状态, 获得最简形式。

- 正则表达式  $\leftrightarrow$  正则文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

- ### ● 关键算法总结:

- ① **Thompson 构造法**: 从正则表达式构造带  $\epsilon$  边的 NFA。
- ② **子集构造法 (Subset Construction)**: 通过  $\epsilon$ -closure 和 *move* 操作将 NFA 确定化。
- ③ **等价类划分法**: 消除 DFA 中的冗余状态, 获得最简形式。

文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

表达式构造带  $\epsilon$  边的 NFA。

**化简 (Minimization)**: 通过  $\epsilon$ -closure 和 *move* 操作将 NFA 确定化并消除冗余状态, 获得最简形式。

等工具如何利用这些原理自动生成高效的词法分析器。

- 正则表达式  $\leftrightarrow$  正则文法  $\leftrightarrow$  NFA  $\leftrightarrow$  DFA  $\rightarrow$  最小化 DFA

- ① **Thompson 构造法**: 从正则表达式构造带  $\epsilon$  边的 NFA。
- ② **子集构造法 (Subset Construction)**: 通过  $\epsilon$ -closure 和 *move* 操作将 NFA 确定化。
- ③ **等价类划分法**: 消除 DFA 中的冗余状态, 获得最简形式。

- **工程实践：**了解 Flex、ANTLR 等工具如何利用这些原理自动生成高效的词法分析代码。

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the founding year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune".

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

# 正则表达式与 NFA 的构造

请使用 **Thompson 构造法**，为下列正则表达式构造等价的非确定有限自动机 (NFA)：

$$r = (a \mid b)^* a (a \mid b)$$

要求：

- 清晰地展示构造的各个子模块（如并运算、闭包运算）。
- 标注出所有的状态编号（从 0 开始）以及  $\varepsilon$  转移边。
- 明确指出开始状态和终态。

## 作业练习：第二题

## NFA 到 DFA 的确定化 (子集构造法)

已知 NFA  $M = (\{0, 1, 2\}, \{a, b\}, \delta, 0, \{2\})$  的状态转移函数  $\delta$  如下:

| 状态 输入 | $a$         | $b$         | $\varepsilon$ |
|-------|-------------|-------------|---------------|
| 0     | $\{0, 1\}$  | $\{0\}$     | $\emptyset$   |
| 1     | $\emptyset$ | $\{2\}$     | $\{2\}$       |
| 2     | $\emptyset$ | $\emptyset$ | $\emptyset$   |

任务：

- 1 请利用 子集构造法 将该 NFA 转换为等价的 DFA。
- 2 详细列出转换过程中的  $\epsilon$ -closure 和  $move$  计算步骤。
- 3 画出（或描述）转换后的 DFA 状态转移图，并标明终态。

## DFA 的状态最小化

给定一个 DFA，其状态集为  $\{A, B, C, D, E\}$ ，开始状态为  $A$ ，终态集为  $\{C, D, E\}$ 。  
转移表如下：

| 状态 | 输入符号 | $a$ | $b$ |
|----|------|-----|-----|
| A  |      | B   | C   |
| B  |      | A   | D   |
| C  |      | E   | C   |
| D  |      | E   | D   |
| E  |      | E   | E   |

### 任务：

- ① 使用等价类划分法对该 DFA 进行最小化。

它中初始划分及每一次迭代后的划分结果